Titulación: Grado en Ingeniería Informática, Ingeniería en Sistemas de Información e InfoAde
Curso: 2025-2026. Convocatoria Ordinaria de Mayo
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 1: Arquitectura PostgreSQL y almacenamiento físico

ALUMNO 1:

Nombre y Apellidos: Jorge Lopez Duque

DNI: 70943373B

ALUMNO 2:

Nombre y Apellidos: José Antonio López González

DNI: 03209889D

Fecha: 02/03/2026

Profesor Responsable: Javier Arrollo Gallardo

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se calificará la asignatura como Suspensa – Cero.

Es obligatorio proporcionar una explicación a lo que está ocurriendo en PostgreSQL cuando así se indica en la cuestión. No solo vale poner un pantallazo. La ausencia de una explicación hará que sea invalidada esa cuestión.

[redacted]

Introducción

En esta primera práctica se introduce el sistema gestor de bases de datos PostgreSQL (18.1 la última). Está compuesto básicamente de un motor servidor y de una serie de clientes que acceden al servidor y de otras herramientas externas. En esta primera práctica se entrará a fondo en la arquitectura de PostgreSQL, sobre todo en el almacenamiento físico de los datos y del acceso a los mismos. Antes de comenzar es obligatorio configurar lo que se comenta en la cuestión. Hay que resolver la práctica con consultas SQL.

Cuestión 0. Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log ("line prefix") aparezcan vuestros DNI's. ¿Cómo se ha realizado la configuración?

La configuración del Logging se ha realizado modificando el fichero postgresql.conf. Se ha activado el logging_collector y se ha configurado el parámetro log_statement con valor mod para que se registren las sentencias SQL DDL y DML generadas.
Asimismo, se ha desactivado el registro de la duración de las consultas y se ha personalizado el prefijo de las líneas del log mediante log_line_prefix, para que se muestren nuestros DNIs.

Finalmente, hemos reiniciado PostgreSQL para aplicar los cambios.

Organización de Archivos en PostgreSQL

Cuestión 1. Crear una nueva Base de Datos que se llame **PL1**. Después crear una tabla **estudiantes** con los siguientes campos:

- estudiante_id: que debe ser un serial empezando por 1.
- nombre: guarda el nombre del estudiante.
- codigo_carrera: guarda el código de carrera del alumno.
- edad: guarda la edad del estudiante.
- indice: guarda el número de créditos superados por el alumno.

Crear un programa que permita generar 30 millones de registros en un fichero de texto que pueda ser cargado posteriormente en la tabla (preferiblemente en Python) con las siguientes propiedades para los siguientes campos, cuyos valores se deben generar aleatoriamente.

- Código de carrera: deben ser valores aleatorios entre 0 y 100.
- edad: deben ser valores aleatorios entre 18 y 40 años (incluidos).
- índice: deben ser valores aleatorios entre 0 y 10000

Cargar los datos en la tabla y localizar los ficheros relacionados con la tabla. ¿cómo se localizan? ¿Cuánto ocupan? ¿por qué?

Postgre almacena las tablas como ficheros heap en el directorio: base/<OID_BD>/<OID_TABLA>.
El tamaño es elevado debido al overhead por fila, el uso de tipos variables como TEXT, los índices creados automáticamente por la clave primaria y los archivos auxiliares FSM y VM necesarios para la gestión interna.

Resultado obtenido:
-Solo tabla: 1.641 MB
-Tabla + índices: 2.284 MB
Por tanto, los índices y estructuras auxiliares ocupan aproximadamente: 2.284 – 1.641 = 643 MB

Los ficheros asociados a una tabla en postgre se localizan a partir de los OIDs de la base de datos y la tabla, dentro del directorio data_directory.
El tamaño total incluye no solo los datos de la tabla, sino también los índices y estructuras auxiliares necesarias para la gestión interna del sistema gestor, lo que explica que ocupe tanto espacio.

Cuestión 2. Calcular teóricamente el tamaño en bloques que ocupa la relación **estudiantes** tal y como se realiza en clase de teoría. ¿Concuerda con el tamaño en bloques que nos proporciona PostgreSQL? ¿Cuál es el factor de bloque medio real de la tabla **estudiantes**? ¿Por qué? Realizar una consulta SQL que obtenga ese valor y comparar con el factor de bloque teórico.

estudiante_id SERIAL -> INTEGER (4 bytes)
nombre TEXT -> suponemos 10 caracteres -> aprox 10 bytes, codigo_carrera INTEGER -> 4 bytes
edad INTEGER → 4 bytes, indice INTEGER → 4 bytes
-Suma total: 4 + 10 + 4 + 4 + 4 = 26 bytes
Postgre usa por defecto: 8 KB = 8.192 bytes por bloque
bfr = 8.192/26 = 315 reg/bloque
Numero teorico de bloques: 30.000.000/315 = 95.238 bloques.

-Tamaño real: 210048 bloques

-Consulta SQL usada:

```
SELECT
   reltuples::bigint AS filas_estimadas,
   relpages AS bloques,
   (reltuples / relpages) AS factor_bloque_medio
FROM pg_class
WHERE relname = 'estudiantes';

relpages = número de bloques
reltuples = número estimado de filas
```

relpages = 210.000 bloques (que viene a ser mas del doble de los teoricos)
Por lo que bfr(real) = 30.000.000/210.000 = 143 reg/bloque.

El tamaño en bloques calculado teóricamente no coincide con el tamaño real proporcionado por postgre, debido a que el modelo teórico no considera el overhead interno por registro, la cabecera de bloque, la alineación de datos y la gestión interna del sistema gestor. El factor de bloqueo medio real es significativamente menor que el teórico.

Cuestión 3. Realizar una consulta que muestre los estudiantes que tengan un índice de 500. ¿Cuántas tuplas se obtienen y cuántos bloques se leen del disco por Postgres? ¿Por qué? Obtener las estadísticas de la tabla y sus campos, para comparar con los resultados obtenidos al aplicar el método visto en teoría. (usar funciones de "The Cumulative Statistics System")

La consulta realiza un escaneo secuencial de la tabla, ya que no existe índice sobre el atributo indice. Postgre debe leer todos los bloques del fichero heap, por lo que el número de bloques leídos coincide aproximadamente con el número total de páginas de la relación.
El número de tuplas obtenidas se aproxima al valor teórico esperado según la distribución uniforme del atributo (aprox 3.000 tuplas).

Las estadísticas obtenidas mediante pg_stat_user_tables y pg_stats permiten comprobar la cardinalidad y la selectividad del atributo, confirmando el comportamiento observado.

shared hit=282 read=209.766

Esta estadistica nos dice que hay 282 bloques en cache y que hemos leido en total 209.766 de disco. Los bloques totales son la suma 210.000 aprox (al no estar ordenado por el parametro indice habra que mirar todos los registrso, es decir, acceder a todos los bloques).

Cuestión 4. Volver a realizar la consulta de la cuestión 3 de nuevo. ¿Cuántas tuplas se obtienen y cuántos bloques se leen del disco por Postgres ahora? ¿Por qué? Comparar con la cuestión anterior

Aunque la consulta se ejecuta por segunda vez, postgre sigue realizando un escaneo secuencial completo. Debido a que la tabla es mayor que la memoria compartida disponible, la mayoría de los bloques deben volver a leerse desde disco. Solo una pequeña parte de los bloques permanece en la caché, lo que explica lao datos que muestran shared hit y shared read.

Buffers: shared hit=309

Esta solo nos registra bloques en la caché y el total es igual a esta cifra porque no hemos leido de disco.

Cuestión 5. Crear una tabla **estudiantes2** cuyas tuplas estén ordenadas físicamente por el campo índice de menor a mayor y que tenga la misma información. Cargar el mismo fichero de datos creado en la cuestión1. Indicar el proceso de generación de dicha tabla ordenada. ¿Cuántos bloques ocupa la tabla ahora? ¿Hay algún cambio? ¿Por qué?

La tabla estudiantes2 se genera utilizando:

```
CREATE TABLE estudiantes2 AS
SELECT *
FROM estudiantes
ORDER BY indice;
```

lo que provoca que las tuplas se escriban físicamente ordenadas por el atributo indice.
El número de bloques ocupados es similar al de la tabla original, ya que el tamaño de los registros no cambia.

Segun postgre esta tabla ocupa 210.048 bloques reales. La ordenación mejora la localidad de acceso pero no reduce significativamente el espacio ocupado.

Cuestión 6. Repetir la cuestión 3 sobre la tabla **estudiantes2** y comparar los resultados obtenidos indicando las conclusiones obtenidas. Relacionarlo con lo visto en teoría.

Aunque la tabla estudiantes2 esté físicamente ordenada por el atributo indice, postgre realiza un escaneo secuencial completo al no existir un índice sobre dicho atributo.
Por ello, el número de bloques leídos es prácticamente el mismo que en la tabla desordenada. Esto se debe a que postgre no utiliza búsqueda binaria sobre el heap, por lo que el orden físico solo mejora la localidad de acceso, lo cual ha sido visible a la hora de ejecutar la sentencia pues ha tardado una menor cantidad de tiempo que antes, pero no reduce el coste del escaneo secuencial.

Estos son los valores que nos arroja el analyze Buffers: shared hit=271 read=209.777

Cuestión 7. Borrar 5000000 tuplas de la tabla **estudiantes** de manera aleatoria usando el valor del campo estudiante_id. ¿Qué es lo que ocurre físicamente en la base de datos? ¿Se observa algún cambio en el tamaño de la tabla y estructuras asociadas a ella? ¿Por qué? Adjuntar el código de borrado.

Al eliminar 5.000.000 de tuplas de forma aleatoria, postgre marca las filas como muertas debido a MVCC. El espacio físico de la tabla y los índices no disminuye inmediatamente, aunque las tuplas ya no sean visibles. El factor de bloque real se reduce y se genera fragmentación interna.
Solo un VACUUM FULL recupera el espacio en disco y compacta la tabla.

-Codigo de borrado:

```
--Borrar filas
DELETE FROM estudiantes
WHERE estudiante_id IN (
   SELECT estudiante_id
   FROM estudiantes
   TABLESAMPLE SYSTEM (30)
   LIMIT 5000000
);
```

Esta instrucción elimina aproximadamente 5 millones de filas de manera "aparentemente aleatoria" usando TABLESAMPLE SYSTEM (30) para elegir bloques al azar. El LIMIT garantiza que no se eliminen más de 5M filas.
Las filas se marcan como muertas y el espacio físico sigue ocupado hasta un VACUUM.
Es una solución eficiente y práctica frente al método clásico ORDER BY random() que sería muy lento.

Cuestión 8. Insertar un nuevo estudiante en la tabla **estudiantes**. ¿dónde se inserta físicamente la nueva tupla? ¿Por qué? (Mirar apartado "System Columns")

Cuando insertamos un nuevo estudiante, postgre lo coloca físicamente en el primer bloque con espacio disponible en el heap.
La ubicación exacta se indica mediante la columna del sistema ctid que identifica el bloque y la posición dentro de ese bloque. Si no hay espacio libre, se asigna un nuevo bloque.
Esto permite reutilizar espacio de filas eliminadas y mantener eficiencia de almacenamiento.

Cuestión 9. En la situación anterior, ¿Qué operaciones se pueden aplicar a la base de datos **PL1** para optimizar el rendimiento de esta? Aplicarlas de tal manera que se recupere el mayor espacio posible. Comentar cuál es el resultado final y qué es lo que ocurre físicamente. (Mirar "Routine Database Maintenance Tasks")

Tras la eliminación masiva de tuplas, la tabla presenta fragmentación interna y múltiples tuplas muertas debido al mecanismo MVCC de postgre.
Para optimizar el rendimiento se pueden aplicar operaciones de mantenimiento como VACUUM, ANALYZE, REINDEX y especialmente VACUUM FULL.
Mientras que VACUUM solo permite reutilizar espacio internamente, VACUUM FULL reescribe completamente la tabla eliminando definitivamente las tuplas muertas, compactando las páginas y reduciendo el tamaño físico en disco. Como resultado final, el número de bloques disminuye, el tamaño en disco se reduce y el factor de ocupación real mejora significativamente.

Ahora la BD ocupa 175.035 bloques despues del vacuum full.

Cuestión 10. Crear una nueva tabla denominada **estudiantes3** con los mismos campos que la cuestión 1 y que esté particionada por el campo código de carrera por medio de la función h = código_carrera mod 20. Insertar los datos del fichero de datos generado en la cuestión 1. Explicar el proceso seguido y comentar qué es lo que ha ocurrido físicamente en la base de datos. ¿Cuándo será útil el particionamiento? ¿Cuántos bloques ocupa cada una de las particiones? ¿Por qué? Comparar con el número bloques que se obtendría teóricamente utilizando el procedimiento visto en teoría. (Ver comando "CREATE TABLE")

Se crea la tabla estudiantes3 particionada por hash sobre el campo codigo_carrera con módulo 20. Postgre distribuye automáticamente cada fila en una de las 20 particiones físicas según el valor de codigo_carrera mod 20.

Físicamente se generan 20 tablas independientes, cada una con su propio heap y bloques.

Dado que los valores están distribuidos uniformemente, cada partición contiene aproximadamente 1.500.000 filas y ocupa alrededor de 1/20 del número total de bloques originales (10.500 aprox).

El particionamiento es especialmente útil cuando las consultas filtran por el campo de partición, ya que permite reducir el número de bloques leídos mediante partition pruning.

Expectativa vs realidad:

Datos:

Número de registros por partición: 30.000.000 / 20 = 1.500.000
Tamaño de cada registro: 26 bytes
Tamaño de bloque: 8.192 bytes

Teoría:
Registros por bloque teórico = 8.192 / 26 = 315
Bloques teóricos por partición = 1.500.000 / 315 = 4.762

Realidad (factor de bloque real = 143 registros/bloque, aprox):
Bloques reales por partición = 1.500.000 / 143 = 10.490

La diferencia entre los bloques teóricos y reales se debe a la cabecera de bloque, la cabecera de fila, padding y el overhead de gestión interna de PostgreSQL

Cuestión 11. Repetir la cuestión 3 sobre la tabla **estudiantes3** y comparar los resultados obtenidos con lo visto anteriormente en las tablas **estudiantes** y **estudiantes2** obteniendo conclusiones sobre el método de partición.

Al repetir la consulta sobre la tabla estudiantes3, postgre realiza un escaneo secuencial sobre cada una de las 20 particiones, ya que el filtro se aplica sobre el atributo indice y no sobre el campo de partición codigo_carrera.
Por tanto, no se produce partition pruning y el número total de bloques leídos es similar al de la tabla original sin particionar.

El particionamiento por hash es útil solamente cuando las consultas filtran por el atributo de partición, ya que permite reducir el número de particiones accedidas y, por tanto, el número de bloques leídos. En este caso, no se obtiene mejora de rendimiento.

El propio analyze nos da estos datos que indican que hemos leido toda la tabla a lo bruto: Buffers: shared hit=13.664 read=197.024

Indexación de PostgreSQL

PostgreSQL soporta indexación definida por el usuario para ayudar a acelerar ciertas consultas. Entre otros tipos de índices soporta árboles y hash. En este apartado se va a trabajar sobre ambos tipos de índices, pudiendo observar cómo se organizan internamente y su funcionamiento. (Mirar módulos "pgstatuple" y "pageinspect")

Cuestión 12. Borrar todas las tablas **estudiantes, estudiantes2** y **estudiantes3**. Crear una nueva tabla que se llama **estudiantes2** como en la cuestión 5 (prdenado por el campo **índice**) y que tenga cargados todos los datos del fichero de texto generado.

Hemos dropeado las tablas con un if exists de proteccion y para crear la nueva hemos cargado los datos del csv y creado el indice.

Cuestión 13. Crear un índice de tipo árbol para **estudiante_id**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos niveles tiene? ¿Cuántos bloques tiene por nivel? ¿Cuántas tuplas tiene un bloque de cada nivel? Indicar el procedimiento seguido e incluir el código SQL utilizado.

El índice arbol B se almacena físicamente como un archivo independiente dentro del directorio de datos de postgre.
Está estructurado en páginas de 8KB organizadas jerárquicamente en niveles (raíz, internos y hojas).
Las páginas hoja contienen las claves indexadas y punteros a las tuplas del heap.
El número de niveles se obtiene mediante la función bt_metap, mientras que el número de bloques y su ocupación se obtiene con pg_relation_size y funciones del módulo pageinspect.

El indice tiene 82.261 bloques y pesa 643Mb. Tiene 1 bloque en la raiz, 291 intermedios y 81.968 hojas estructurados en 3 niveles. Hay 367 tuplas por bloque por nivel.

-Codigo utilizado:

```
--Crear incice arbol B
CREATE INDEX idx_estudiantes2_id
ON estudiantes2(estudiante_id);
```

(El arbol B es el indice por defecto que hace postgre, por eso de momento no hace falta especificarlo)

Cuestión 14. Determinar el tamaño de bloques y el número de niveles que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 13.

Tamaño entrada = 4 (estudiante_id) + 6 (TID) = 10 bytes

Entradas por bloque: 8.192/10 = 819 entradas por bloque.

Bloques hoja, con N=30.000.000 tuplas: 30.000.000/819 = 36.631 bloques hoja.

Nivel interno: 36.631/819 = 45 bloques internos.

Raíz 45/819 = 1 bloque raíz.

Altura total: 3 niveles.

El modelo teórico predice 819 entradas por bloque y 36.631 hojas.
Sin embargo, el índice real presenta 367 entradas por bloque y 81.968 hojas.
La altura del árbol coincide (3 niveles), pero el número de bloques es más del doble.
La diferencia se debe a que el modelo teórico no considera overhead de página, alineación, fillfactor ni fragmentación interna. PostgreSQL no utiliza el 100% del bloque para datos útiles, lo que incrementa significativamente el número de páginas necesarias.

Cuestión 15. Crear un índice de tipo hash para el campo **estudiante_id**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos cajones tiene? ¿Cuántas tuplas tiene de media un cajón? Indicar el procedimiento seguido e incluir el código SQL utilizado.

El índice hash se almacena físicamente como un archivo independiente dentro del directorio de la base de datos. Está organizado en cajones en lugar de niveles jerárquicos. El número de cajones se obtiene mediante la función hash_metapage_info. El número medio de tuplas por cajones se calcula dividiendo el número total de registros entre el número de cajones A diferencia del arbol B no soporta búsquedas por rango y no tiene estructura en niveles.

El indice pesa 1022Mb, tiene 130.774 bloques, 98304 cajones (98303 del maxbucket +1), hay de media unas 299 tuplas por cajon.

```
--Tamaño y numero de bloques
SELECT
  pg_size_pretty(pg_relation_size('idx_estudiantes2_id_hash')) AS tamaño,
  pg_relation_size('idx_estudiantes2_id_hash')/8192 AS bloques;

--Numero de cajones
CREATE EXTENSION IF NOT EXISTS pageinspect;

SELECT *
FROM hash_metapage_info(get_raw_page('idx_estudiantes2_id_hash', 0));
--num_buckets (el numero de cajones) = maxbucket + 1

-Codigo para crear el indice:
--Crear indice hash
CREATE INDEX idx_estudiantes2_id_hash
ON estudiantes2 USING HASH (estudiante_id);

(Le especificamos que cree un indice de tipo hash usando el campo estudiante_id)
```

Cuestión 16. Determinar el tamaño de bloques y el número de cajones que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 15.

Tamaño entrada = 4 (estudiante_id) + 6 (TID) = 10 bytes.

Entradas por bloque: 8.192/10 = 819 entradas por bloque.

Número de cajones necesario: 30.000.000/819 = 36.631 cajones. (Implica que ocupa 36.631 bloques)

El modelo teórico predice 36.631 cajones con 819 tuplas por cajon. Sin embargo, el índice real presenta 98.304 cajones con una media de 299 tuplas por cajon.
El tamaño real es aproximadamente 2,7 veces superior al teórico.

La diferencia se debe a que enel modelo teórico no consideramos overhead interno, gestión dinámica del hashing, páginas de overflow ni reserva de espacio libre. PostgreSQL no utiliza el 100% del bloque para datos útiles, lo que incrementa significativamente el número de páginas necesarias.

Cuestión 17. Crear un índice de tipo árbol para el campo **índice**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos niveles tiene? ¿Cuántos bloques tiene por nivel? ¿Cuántas tuplas tiene un bloque de cada nivel? Indicar el procedimiento seguido e incluir el código SQL utilizado.

El índice arbol B sobre el campo indice se almacena físicamente como un archivo independiente dentro del directorio de datos de postgre.
Está organizado jerárquicamente en páginas de 8KB divididas en raíz, páginas internas y hojas.
El número de niveles se obtiene mediante la función bt_metap, mientras que el número de bloques y su distribución por nivel se obtiene mediante funciones del módulo pageinspect.
Las páginas hoja contienen las claves indexadas y los punteros a las tuplas del heap.

Dado que la tabla estaba previamente ordenada por el campo indice, el índice presenta una buena correlación física con la tabla.

El indice tiene 25.910 bloques y pesa 202Mb. Tiene 1 bloque en la raiz, 126 intermedios y 25.782 hojas estructurados en 2 niveles. Hay 10 tuplas por bloque por nivel.

El codigo utilizado para la creacion de este indice es el mismo que el de la cuestion 13, pero cambiando estudiante_id por indice.

Cuestión 18. Determinar el tamaño de bloques y número de niveles que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 17.

Tamaño entrada = 4 (indice) + 6 (TID) = 10 bytes

Entradas por bloque: 8.192/10 = 819 entradas por bloque.

Bloques hoja, con N=30.000.000 tuplas: 30.000.000/819 = 36.631 bloques hoja.

Nivel interno: 36.631/819 = 45 bloques internos.

Raíz 45/819 = 1 bloque raíz.

Altura total: 3 niveles.

El índice arbol B creado sobre el campo indice, que coincide con el orden físico de la tabla, presenta un tamaño total de 202 MB y ocupa 25.910 bloques (1 raíz, 126 intermedios y 25.782 hojas), estructurados en 2 niveles.
Comparado con el modelo teórico (36.631 hojas y 3 niveles), el índice real presenta menos bloques hoja y menor altura del árbol. Esto indica una mayor compactación de las páginas.

La razón principal es que las inserciones se realizaron de forma secuencial al estar la tabla ordenada por dicho campo. Esto reduce significativamente los page splits y la fragmentación, permitiendo una ocupación más eficiente de las páginas hoja.
En consecuencia, el árbol resultante es más pequeño, más compacto y requiere menos niveles para acceder a cualquier registro, lo que mejora el rendimiento.

Cuestión 19. Crear un índice de tipo hash para el campo **índice**. ¿Dónde se almacena físicamente ese índice? ¿Qué tamaño tiene? ¿Cuántos bloques tiene? ¿Cuántos cajones tiene? ¿Cuántas tuplas tiene de media un cajón? Indicar el procedimiento seguido e incluir el código SQL utilizado.

El índice hash sobre el campo indice se almacena físicamente como un archivo independiente dentro del directorio de datos de postgre.
Está organizado en cajones en lugar de niveles jerárquicos.
El número de cajones se obtiene mediante la función hash_metapage_info, mientras que el tamaño y número de bloques se calculan mediante pg_relation_size.

La media de tuplas por cajón se obtiene dividiendo el número total de registros entre el número de cajones.

Debido a la repetición de valores en el campo indice, pueden existir colisiones y páginas overflow.

El indice pesa 1.315Mb, tiene 168.337 bloques, 98.304 cajones (98.303 del maxbucket +1), hay de media unas 299 tuplas por cajon.

```
--Tamaño y numero de bloques
SELECT
  pg_size_pretty(pg_relation_size('idx_estudiantes2_indice_hash')) AS tamaño,
  pg_relation_size('idx_estudiantes2_indice_hash')/8192 AS bloques;

--Numero de cajones
CREATE EXTENSION IF NOT EXISTS pageinspect;

SELECT *
FROM hash_metapage_info(get_raw_page('idx_estudiantes2_indice_hash', 0));
--num_buckets (el numero de cajones) = maxbucket + 1
```

El codigo utilizado para la creacion de este indice es el mismo que el de la cuestion 15, pero cambiando estudiante_id por indice.

Cuestión 20. Determinar el tamaño de bloques y número de cajones que teóricamente tendría de acuerdo con lo visto en teoría. Comparar los resultados obtenidos teóricamente con los resultados obtenidos en la cuestión 17.

Tamaño entrada = 4 (indice) + 6 (TID) = 10 bytes.

Entradas por bloque: 8.192/10 = 819 entradas por bloque.

Número de cajones necesario: 30.000.000/819 = 36.631 cajones. (Implica que ocupa 36.631 bloques)

El índice HASH creado sobre el campo indice ocupa 1.315 MB y está compuesto por 168.337 bloques, con 98.304 cajones y una media de 299 tuplas por cajón.
A diferencia del índice arbol B, el hecho de que la tabla esté físicamente ordenada por el campo indice no influye en la estructura del índice HASH. Esto se debe a que el hashing aplica una función de dispersión sobre la clave, destruyendo completamente el orden natural de los datos.
Como consecuencia, la distribución de las tuplas en los cajones es independiente del orden físico de la tabla, y el número de cajones y la ocupación media coinciden prácticamente con los obtenidos para el hash sobre estudiante_id.
El tamaño del índice es considerablemente mayor que el modelo teórico debido a que este no contempla overhead interno, páginas de desbordamiento ni la gestión dinámica del hashing implementada por PostgreSQL.

Se confirma así que los índices HASH no se benefician del orden físico de la tabla, ya que su organización depende exclusivamente de la función hash aplicada sobre la clave.

Cuestión 21. ¿Qué conclusiones se puede obtener de la gestión y organización de PostgreSQL sobre los dos índices árbol y hash que se han creado y han sido analizados? ¿Por qué? Comparar con lo visto en teoría.

Del análisis realizado sobre los índices arbol B y HASH se pueden extraer varias conclusiones importantes sobre la gestión y organización interna de PostgreSQL.

En primer lugar, el índice arbol B muestra una fuerte dependencia del patrón de orden. Cuando se crea sobre un campo desordenado (como estudiante_id), el árbol presenta mayor número de bloques hoja, mayor tamaño total y más fragmentación debido a los frecuentes page splits.
Sin embargo, cuando el índice se crea sobre un campo por el que la tabla está físicamente ordenada (campo indice), el árbol resultante es mucho más compacto, con menos bloques hoja, menor tamaño y menor altura.
Esto confirma que los arboles B son estructuras ordenadas y su eficiencia depende del orden de los datos.

En segundo lugar, el índice HASH no se ve afectado por el orden físico de la tabla. Tanto para estudiante_id como para indice, el número de cajones y la ocupación media por cajon son prácticamente iguales. Esto ocurre porque la función hash destruye el orden natural de los valores y distribuye las tuplas según el resultado de dicha función. Por tanto, el rendimiento y la estructura del índice HASH dependen exclusivamente de la función de dispersión y no del patrón de inserción.
Además, en todos los casos se observa una diferencia significativa entre el modelo teórico y los resultados reales.
El modelo teorico asume bloques completamente llenos y ausencia de sobrecarga interna, mientras que postgre reserva espacio para metadatos, mantiene factores de llenado inferiores al 100% y gestiona dinámicamente la estructura de páginas. Esto provoca que los índices reales ocupen más bloques y más espacio del estimado teóricamente.

En conclusión, el estudio confirma que el arbol b es más versátil y se beneficia claramente del orden físico de los datos, el HASH es independiente del orden, pero puede ocupar más espacio y no soporta búsquedas por rango y que la implementación real de postgre introduce sobrecarga y mecanismos de gestión que se apartan del modelo teórico ideal, aunque mantienen la misma estructura conceptual.

Hemos comprobado que hay cosas para las que una estructura es superior a la otra en ambos casos pero que en la realidad ambas requieren pensamiento crítico para saber donde usarlas pese a lo guiado de la practica. Suponemos que a nivel de diseño en el mundo real habrá que tomar incluso mas consideraciones que en la práctica. Basicamente es cuestion de analisis situacional y eficiencia de recursos.

Monitorización de la actividad de la base de datos

En este último apartado se mostrará el acceso a los datos con una serie de consultas sobre la tabla original. En este apartado se pretende mostrar cómo es el acceso a los datos para diferentes tipos de consultas.

PostgreSQL suministra varias vistas estadísticas que se pueden usar para monitorizar los bloques leídos (tipo statio de la sección The Cumulative Statistics System) de cada una de las estructuras creadas en la base de datos. En este apartado se deben usar esas vistas y está prohibido el uso de otro comando para este fin (Table 28.2).

Para ello, borrar todas las tablas creadas y volver a crear la tabla **estudiantes** como en la cuestión 1. Cargar los datos que se encuentran originalmente en el fichero generado en la cuestión 1.

Cuestión 22. Crear un índice primario tipo árbol sobre el campo **índice**, y otro sobre **código_carrera**. También crear un índice hash sobre el campo **estudiante_id** y otro sobre **índice**. ¿Cuál ha sido el proceso seguido para crear cada tipo de índice? Incluir el código SQL utilizado para ello.

```
CREATE INDEX nombre ON tabla USING hash/btree(campoTabla);

Este es un comando ejemplo (si no no me da el espacio), la idea es que Postgre va recorriendo la tabla
fijandose en el campo que le hemos pasado en campoTabla y ya si estamos hablando de un hash, lo mete en
el bucket correspondiente. Si hablamos de un arbol lo que hace es ir construyendolo y metiendo entrada
por entrada donde corresponde. (Al final si entra)

CREATE INDEX arbolIndice ON estudiantes USING btree(indice);
CREATE INDEX arbolCodigoCarrera ON estudiantes USING btree(codigo_carrera);
CREATE INDEX hashId ON estudiantes USING hash(estudiante_id);
CREATE INDEX hashIndice ON estudiantes USING hash(indice);
```

Cuestión 23. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántas tuplas se han obtenido? ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Comparar con lo visto en teoría. Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta. Se recuerda que solo se pueden usar vistas sobre las estadísticas de la base de datos (usar funciones de "The Cumulative Statistics System").

1. Mostrar la información de las tuplas con código de carrera igual 50.

En esta consulta hemos encontrado todos los elementos con codigo de carrera igual a 50, la consulta ha hallado 297.000 tuplas y segun las estadisticas se han leido 253 bloques de indice y 258 de tabla; ambos siendo hits de caché.

La consulta la hemos realizado primero reiniciando las estadisticas para ver bien que está pasando, segundo con esta sentencia SQL: `SELECT * FROM estudiantes WHERE codigo_carrera = 50;` y tercero consultando en pg_stat_all_tables y pg_statio_all_tables.

Los datos sugieren que se leyó el arbol y de ahí se mando con los punteros a la tabla. Por eso tenemos coincidencias tanto en tabla como en indices.

Estos resultados tienen sentido por como estan sucediendo las cosas. Primero en vez de leer toda la tabla secuencialmente estamos bajando por el arbol hasta encontrar los que nos cumplen la condicion y ya de ahi estamos leyendo en tabla lo que necesitamos (aunque en este caso caché).

2. Mostrar la información de la tupla con estudiante_id igual a 80000.

Esta consulta se hace con el siguiente SQL: SELECT * FROM estudiantes WHERE estudiante_id = 80000; de aquí y las estadisticas podemos ver como lo ha encontrado.

En total se han leido 4 bloques de disco. 3 del indice hash y 1 de la tabla para dar una única tupla resultado.

Esto esta funcionando de la siguiente manera. El SGBD está calculando el hash de 80000 y encuentra esos buckets, de ahí se leen esos 3 bloques y se llega al que nos interesa en la tabla.

3. Mostrar los datos de los estudiantes que tienen un índice entre 100 y 200.

Se han hallado 302000 tuplas y se ha usado esta sentencia: SELECT * FROM estudiantes WHERE indice BETWEEN 100 AND 200; con las estadisticas vemos que se han leido 256 bloques por el arbol de los indices y luego en tabla se han leido otros 336. Todo de disco.

Los numeros cuadran y lo que se hace es llegar al valor minimo bajando por el arbol y luego recorrerlo secuencialmente hasta llegar al valor maximal para obtener todos los punteros que nos llevaran a los bloques que tengan los elementos que cumplen esta condición.

4. Contar el número de estudiantes que tienen una edad de 20 años.

Sabemos que sobre "edad" no tenemos ninguna estructura de busqueda, asi que lo vamos a tener que hacer de manera secuencial. Esta misma teoria es la que nos reflejan las estadisticas. Hemos encontrado 1304000 tuplas y segun estadisticas tenemos 166 bloques leidos de disco y 11 de caché. Dando un total de 177 bloques totales.

Esta es la sentencia usada: SELECT * FROM estudiantes WHERE edad = 20;

El resultado es completamente esperado y no se sale del marco teorico.

5. Mostrar el número total de estudiantes que tienen el mismo índice.

Tenemos 10000 indices coincidentes, para esta consulta se han leido 194000 bloques de disco y 16000 de cache (toda la BD) para poder organizarlos por indice coincidente.

La consulta ha sido esta: SELECT indice, *COUNT*(*) FROM estudiantes GROUP BY indice;
Esto tiene sentido porque para este tenemos que leer si o si todos, y ademas de manera secuencial. Porque aunque tengamos sobre este campo arbol y hash no tiene excesivo sentido usar uno u otro porque el hash seria leerlos secuencialmente pero con los buckets y con el arbol nos pasa lo mismo, es leer el arbol secuencial. Al haber que leerlos todos para hacer las comparaciones lo mas eficaz es hacerlo a lo bruto.

6. Obtener la media de edad de los estudiantes que tienen la misma carrera.

Vamos a ver la consulta SQL y luego ya nos metemos en que pasa. Para este ejercicio sería esta:

SELECT codigo_carrera, *AVG*(edad) FROM estudiantes GROUP BY codigo_carrera;

Por partes, esto hace una media de unos datos, nos interesa como encontramos esos datos que es lo que va a leer bloques, lo otro es matematica. Vemos que tenemos que recorrer todos los bloques de la tabla asi que teoricamente seran los del ejercicio anterior, ademas, antes hemos visto que lo ha hecho de manera secuencial.

Si comprobamos las estadisticas tenemos exactamente eso, que de los 101 codigos de carrera ha ido haciendo las medias (29 años en todas aprox) y que todos los accesos a bloques han sido sequenciales (con los mismos en cache y disco de antes)

7. Insertar un nuevo estudiante con un índice de 100.

En las estadisticas vemos que se accede a 4 bloques en disco y en los indices todos acceden a mas o menos 4 menos el hash por indice que accede a 17 bloques entre caché y memoria.

En general hemos tenido una buena situacion porque no se ha tenido que tocar mucho para almacenar el nuevo registro.
A las malas habria que hacer un split de pagina en los arboles o un desbordamiento de bucket en el hash. Por eso vemos que no se modifica mas que lo que es imprescindible, no hay que rehacer todo.

Para insertar a crazy dave hemos usado esta sentencia:

```
INSERT INTO estudiantes(estudiante_id, nombre, codigo_carrera, edad, indice)
VALUES (30000002, 'Crazy Dave', 7, 30, 100)
```

Cuestión 24. Borrar los índices creados en la cuestión 20. Crear un índice multiclave tipo árbol sobre los campos **código_carrera e índice**. Incluir el código SQL utilizado para ello.

Borro y creo:

```
DROP INDEX IF EXISTS arbolindice;
DROP INDEX IF EXISTS arbolcodigocarrera;
DROP INDEX IF EXISTS hashid;
DROP INDEX IF EXISTS hashindice;

CREATE INDEX ON estudiantes USING btree(codigo_carrera, indice);
```

Esto hace un arbol ordendado primero por codigo carrera y luego dentro de cada codigo, por indice.

Cuestión 25. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta:

1. Mostrar los estudiantes que tienen un código de carrera de 20 y un índice de 500.

Se ha usado este SQL: `SELECT * FROM estudiantes WHERE codigo_carrera = 20 AND indice = 500;`

Además, esta estructura lo que está haciendo es ordenar por codigo carrera y para codigo carrera ordena por indice, lo que hace esta consulta muy rapida porque bajamos por el arbol hasta la seccion codigo carrera = 20 y de ahí podemos hacer una busqueda binaria porque el dato está ordenado. Con los punteros que se consiguen luego se va a tabla y se recupera lo que se buscaba.

Las estadisticas me estan mostrando 4 bloques de arbol y 25 de tabla, como lo he ejecutado dos veces los de tabla los tengo en cache, pero la primera vez era todo disco. Hemos recuperado 25 tuplas.

2. Mostrar los estudiantes que tienen un código de carrera de 50 o un índice de 900.

Se ha usado este SQL: `SELECT * FROM estudiantes WHERE codigo_carrera = 50 OR indice = 900;`

Esta busqueda Postgre ha decidido hacerla por secuencial. Esto es porque la esta analizando como "dos busquedas distintas" los que tengan codigo = 50 y luego ya los de indice = 900 (y quitar duplicados). Encontrar los de codigo = 50 se hace maravillosamente con este arbol, pero los de indice 900 no. Esto es por que el arbol primero se ordena por codigo, y luego de ahí por indice, pero imaginemos que en el codigo n tenemos los indices 1, 4, 6, 7. Nada nos impide que en el codigo n+1 tengamos 1, 4, 24, 73. No estan ordenados en la gran escala, solo en la local.

Lo que hace el Postgre es decir: "Para que voy a leer por el arbol si luego me va a tocar volver a leer la tabla entera, voy a leer la tabla y ya.". Esto se refleja en las estadisticas, saliendo que son todas leidas secuenciales donde 477 bloques se han leido de disco y 16 de cache (los del ejercicio anterior que se hubiesen quedado en caché). Hemos obtenido 300000 tuplas.

3. Mostrar los estudiantes que tienen un índice de 300.

Se ha usado este SQL: SELECT * FROM estudiantes WHERE indice = 300;

Mismo problema de antes y misma solución, leer secuencialmente todo. Exactamente lo que nos indican las estadisticas, que se han leido 35200 + 22 bloques entre disco y caché y que todos han sido secuenciales. Se han hallado 3000 resultados

4. Mostrar los estudiantes que tienen un código de carrera de 80.

Se ha usado este SQL: SELECT * FROM estudiantes WHERE codigo_carrera = 80;

Este con nuestro arbol de ahora maravilloso, es como si lo hiciesemos con un arbol simple. Es bajar, buscar, mirar punteros y a tabla. Rapido y eficiente con nuestro indice actual. Se han hallado 298000 tuplas y se han leido todos por indice 274 + 1 bloques del arbol y 204 + 122 de tabla (siendo ambos entre disco y caché).

Cuestión 26. Crear la tabla **estudiantes3** particionada por el campo **edad** de tal manera que los estudiantes que tengan la misma edad estén juntos en la misma partición. Para cada una de las consultas que se muestran a continuación, ¿Qué información se puede obtener de los datos monitorizados por la base de datos al realizar la consulta? Comentar cómo se ha realizado la resolución de la consulta. ¿Cuántos bloques se han leído de cada estructura? ¿Por qué? Comparar con la teoría. Importante, reinicializar los datos recolectados de la actividad de la base de datos antes de lanzar cada consulta.

1. Mostrar el número de estudiantes con una edad de 25.

Usando este SQL: SELECT * FROM estudiantes3 WHERE edad = 25;
Vemos que nos devuelve 6 bloques leidos en memoria, el de buscar la tabla del 25 y 5 en la propia tabla (que salen en caché porque ya lo he tirado, deberia salir en disco) obteniendo 1300000 tuplas. Esto es maravillosamente eficiente para esta consulta (y las siguientes) porque es acceder solo a la tabla que nos interesa y practicamente devolverla entera. Esto se llama partition pruning y ademas salen tan pocos bloques porque los hemos metido todos a la tabla y no hemos movido para borrar/añadir.

2. Mostrar los estudiantes que tienen una edad de 20 o 25 o 30 años.

Hemos usado este SQL: SELECT * FROM estudiantes3 WHERE edad = 20 OR edad = 25 OR edad = 30;

Y aquí tenemos problemas con las estadisticas porque solo aparece que se lean bloques en la tabla del 20. En principio se estan devolviendo resultados de todas las edades que buscamos. Segun lo que he leido en internet es por el propio postgres que lo que hace es lanzar hilos que no se suman bien a la estadistica, y el hilo principal solo hace la primera parte de la consulta. Con el explain se deberia ver bien, pero nos ceñimos a este.

Segun las estadisticas se leen 6 bloques, 5 en la propia tabla y 1 para encontrar la tabla como tal. Lo que hace la consulta es ir a la tabla del 20, darla como solucion, ir a la tabla del 25, añadirla, ir a la del 30, añadirla tambien.

3. Mostrar los estudiantes que tienen una edad entre 25 y 30 años (incluidos).

Mismo problema de la anterior y misma forma de pensar, solo se leen los 6 bloques de la primera tabla, los demas no se reflejan.

Este es el SQL usado: SELECT * FROM estudiantes3 WHERE edad BETWEEN 25 AND 30;

Se han recuperado 7824938. Segun el datagrip no me ha dejado ir al final de la tabla resultados sin alocar mas memoria al proceso. Entonces para hallar el numero hemos usado un COUNT(*) en el sql de arriba.

4. Mostrar los estudiantes que tienen una edad mayor de 50 años.

Hemos usado este sql: SELECT * FROM estudiantes3 WHERE edad > 50;

Esta es muy curiosa, como sabemos que las edades estan acotadas hasta los 40 no hemos creado ni la paginacion para 50 o mas ni para default. Lo que hace es no leer nada porque ninguna encaja y no devolver nada porque no hay ninguno.

Cuestión 27. A la vista de los resultados obtenidos de este apartado, comentar las conclusiones que se pueden obtener del acceso de PostgreSQL a los datos almacenados en disco.

Creo que lo mas importante que se refleja aqui es que hay que pensar bien las cosas para hacer una base de datos. Si por ejemplo fuese un banco con muchos clientes para buscar cada cuenta no usaria un arbol, si no un hash porque "no se usan" consultas de rangos. Si por otra parte fuesemos una agencia de publicidad donde necesitamos rangos usaria arboles. No es tanto que las bases no esten bien hechas si no que los indices que se usan o se dejan de usar pueden no ser optimos para los que se necesita.

Tomando esto de la paginación, si fuese una gran empresa internacional por ejemplo me podria interesar paginar por paises para que no halla que consultar todos los datos a la hora de hacer una chorrada de consulta.

En resumen, la eficiencia esta mas en saber que usas, como lo usas y por qué lo usas que en hacerlo a la fuerza bruta y simplemente mejorar hardware.

Y sobre lo de PostgreSQL (por si acaso no he entendido bien la pregunta) hemos aprendido que con las estadisticas cumulativas si los "hilos" se usan en gran cantidad la estadisticas no salen bien porque solo se refleja el hilo principal.

Bibliografía (PostgreSQL 18.1)

- Capítulo 1: Getting Started.
- Capítulo 5: 5.6 System Columns.
- Capítulo 5: 5.12 Table Partitioning.
- Capítulo 11: Indexes.
- Capítulo 19: Server Configuration.
- Capítulo 24: Routine Database Maintenance Tasks.
- Capítulo 27: Monitoring Database Activity. The statistics Collector
- Capítulo 27.6: Monitoring Disk Usage. Determining Disk Usage
- Capítulo 52: System Catalogs
- Capítulo 65.1: B-Tree Indexes
- Capítulo 65.6: Hash Indexes
- Capítulo 67: Database Physical Storage
- Capítulo VI.II: PostgresSQL Client Applications.
- Capítulo VI.III: PostgresSQL Server Applications.
- Apéndice F: Additional Supplied Modules. Pageinspect, pgstatutuple
- Apéndice G: Additional Supplied Programs. Oid2name